* Unit-2 Greedy Method * & Dynamic Programming

* **Greedy Method** Choose the best option available right now.

Greedy method is an Algorithm design techniques that constructs a solution step by step by selecting the best feasible choice at each stage, with the aim of obtaining a globally optimal solution.

Ex: fractional knapsack, job scheduling (sequencing), optimal merge pattern, minimum spinning tree.

Knapsack Problem -

It is an optimization problem in which we selects an item in such a way that -

→ The total weight does not exceed in knapsack problem.

→ The total profit is maximum.

ratio = $\frac{P_i}{W_i}$

Types
- fractional knapsack → An item can be devided into smaller parts
- 0/1 Knap sack → An item can not be devided into smaller parts.

We have two choices.

0 → item not selected

1 → whole item is selected.

Note: 0/1 knapsack is part of dynamic programming

eg: let suppose we have

I	I_1	I_2	I_3	I_4	I_5
w	5	10	20	30	40
v	30	20	100	90	160

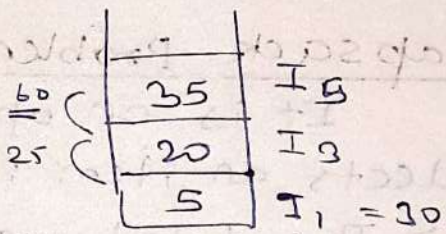
} key.

Capacity of knapsack is given as 60kg.

profit	I_1	I_2	I_3	I_4	I_5
w	5	10	20	30	40
v	30	20	100	90	160
v/w	6	2	5	3	4

in ascending order

	w	v
I_1 → 6	5	30
I_3 → 5	20	100
I_5 → 4	40	160
I_4 → 3	30	90
I_2 → 2	10	20



40 → 160

1 → 160/40

35 → 160/40 × 35 = 140

values = $V(I_1 + I_4 + I_5)$

= 30 + 100 + 140

= 270 Ans

Algorithm

1. Calculate Profit/weight ratio

2. FractionKnapSack (item, W)
   for ratio each item
   ratio = profit/weight

   Sort item by ratio in descending order
   totalpoft = 0

   for each item:
   if item.weight <= W;
   W = W - item.weight
   totalprofit = totalprofit + item.profit

   else:
   fraction = W / item.weight
   totalprofit = totalProfit + item.profit * fraction

   break
   return totalProfit

Java Code

```
import java.utils.Arrays;
import java.utils.Comparator;
pwblic class main{
    Static class Item{
        int weight;
        int profit;
    Item(int weight, int profit){
        this.weight = weight;
        this.weight = profit;
    }
```

```
int n = 3;
int capacity = 50;
double maxProfit = fractionalKnapSack(
                    items, n, capacity);
System.out.println("Maximum Profit = "+
                    maxProfit);
    }
}
```

Time Complexity

$O(n \log n)$

Space $\Rightarrow O(1)$ auxiliary space.

* Job Sequencing

In the algorithm, we schedule job in such a way,

1) Total profit should be maximum.
2) every slected job is completed before or on its deadline.
3) ex: Each job requires exactly one unit of time.

Deadline

That means a problem/ task should be completed by time slot

So steps

1. Sort jobs by decreasing profit
2. Take highest profit job.
3. Put it in latest available slot before its deadline
4. Continue with next highest profit Job

Why Latest slot.

Ex: slot 1 2 3

J_1 ↗ then (2 & 3 early)

if we put it into another slot then it will occupy unnecessarily early slot.

Note: Place each selected job as late as possible while still meeting its deadline.

Ex:

Job	Deadline	Profit
J_1	2	100
J_2	1	50
J_3	2	200
J_4	1	40
J_5	3	150

maximum deadline 3

So we have slot 1 2 & 3

Sort by profit

Job	Deadline	Profit
J_3	2	200
J_5	3	150
J_1	2	100
J_2	1	50
J_4	1	40

Step 2 Schedule Lightest profit

J_3

slot 1	slot 2	slot 3
	J_3	

profit = 200

Now J_5 schedule.

Deadline = 3 Latest available slot ≤ 3

slot 3

Slot 1	Slot 2	Slot 3
	J_3	J_5

Total profit : 200 + 150

Now J_1 schedule

Deadline = 2 Latest available slot ≤ 2

slot = 2 already occupied by J_3

So slot 1

Slot 1	Slot 2	Slot 3
J_1	J_3	J_5

Total profit: 200 + 150 + 100

Now all slot are occupied So we cannot ~~add~~ en schedule any Job

final Total profit = 200 + 150 + 100

Maximum profit: 450

Time Complexity → $O(n \log n + n^2)$

→ $O(n^2)$

* Optimal merge pattern :-

Optimal merge pattern is a greedy techniques used to merge multiple sorted files into a single file with minimum total computation/merge cost. At each step, the two files with the smallest sizes are merged first.

Algorithm

1. Insert all files sizes into a min Heap
2. total cost = 0
3. While more than one files remains :
   - Remove smallest file x.
   - Remove second smallest file y
   - merged = x + y
   - totalCost += merged
   - Insert merged back into min Heap.
4. Return total cost.

Note always merge two smallest cost files.

Ex 5 10 20 30

Step 1

= 5 + 10 = 15

15 20 30

35 30

65

Now total

⇒ 15 + 35 + 65

⇒ 115

(minimum merge cost)

We use min Heap, that keeps the smallest element easily accessible.

Code:

```
int totalCost = 0;
while (minHeap.size() > 1) {
    int first = minHeap.top();
    minHeap.pop();
    int second = minHeap.top();
    minHeap.pop();
    int merged = first + second;
    totalCost += merged;
    minHeap.push(merged);
}
return totalCost;
```

Complexity: $O(n \log n)$ = Time Complexity

Space Complexity = $O(n)$

Note: $(n-1)$ merge occurs for n files.

* Huffman Optimization Method:- (Variable Coding length of

→ Always select two symbols/nodes having the smallest frequencies.

Huffman coding is a greedy algorithm for constructing an optimal prefix-free binary code from symbol frequencies. It repeatedly combines the two least-frequent symbols & inserts their combined frequency back into a priority queue until a single tree is formed.

$$\text{Total Cost} = \sum_{i=1}^{n} f_i \times l_i$$

l = length of code.

Algorithm

1. Calculate frequency of every character.
2. Create leaf node for each character.
3. Insert all node into a MinHeap according to frequency.
4. While more than one node remains:
   - Remove two nodes having smallest freq.
   - Create a new internal node.
   - new freq = freq1 + freq2
   - make two child node of new node.
   - Insert new note back into min Heap
5. Left → 0 Right → 1

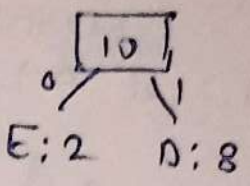
Ex =

Char	A	B	C	D	E	F
freq.	30	15	25	8	2	20

Step 1 keep in ascending order of freq.

Char	E	D	B	F	C	A
freq.	2	8	15	20	25	30

Select two smallest



merge = 2 + 8 = 10

New subtree weight = 10

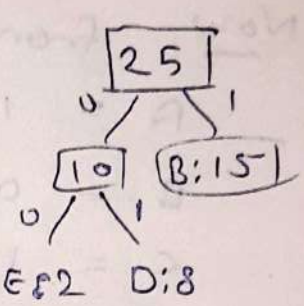
Step 2 =

B F C A

10 15 20 25 30

merge = 10 + 15 = 25

subtree weight = 25

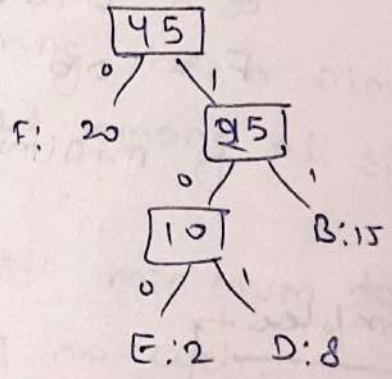


Step 3 =

F C A

20 25 25 30

merge = 45



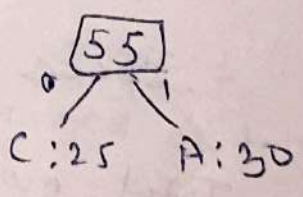
New subtree weight = 45

Step 4 =

C A

25 30 45

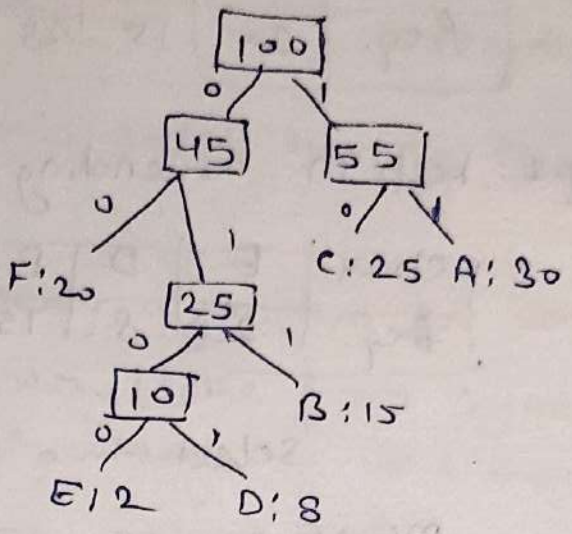
merge



new subtree weight = 55.

Step 5 45 55

merge = 100



Final Heap tree.

Now from root, find bits.

A = 11 => 2 × 30 = 60

B = 011 => 3 × 15 = 45

C = 10 => 2 × 25 = 50

D = 0101 => 4 × 8 = 32

E = 0100 => 4 × 2 = 8

F = 00 => 2 × 20 = 40

minimum frequency 235

Complexity

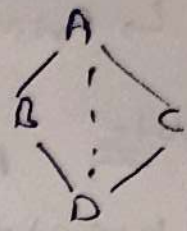
Time Complexity = O(n log n)

space complexity = O(n)

Total merge = n − 1.

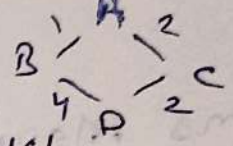
* Minimum Spinning Tree (MST) :-

Graph consists of vertices (nodes)
edges (connections)

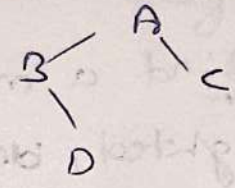


vertices (A, B, C, D)

weighted graph have weight/cost attached with edge.



→ Tree => Connected graph with no cycle.



V => vertecs
then V-1 edges.

A graph can have multiple spinning tree.

MST => A spinning tree having the minimum possible total edge weight among all spinning tree of the graph.

→ Connect all vertices with minimum total edge weight with having no cycle.

So => Choose edges such that all vertices becomes connected, no cycle is created & total weight is minimum.

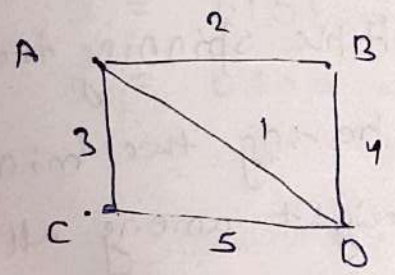
Algorithm used for MST
- Kruskal Algorithm
- Prim's Algorithm

Note: A minimum spanning Tree of a connected, weighted, undirected graph is a spanning tree whose total edge weight is minimum among all possible spanning tree of the graph.

* Prims Algorithm

used to find a minimum spinning tree of a connected, weighted, undirected. graph.

Rule: Choose minimum weight edge crossing from MST.



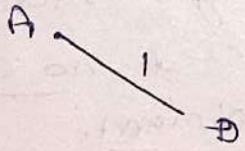
AB = 2
AC = 3
AD = 1
BD = 4
CD = 5

we have V = 4.

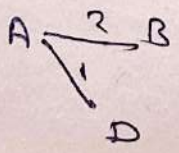
must contain = V-1 = 3 edges.

So Start from A

AD min

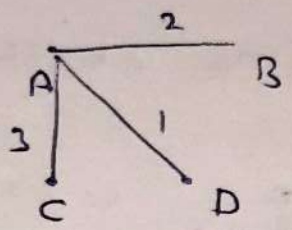


A→B min = 2



Now from C

A-C = 5 min



So 1 + 2 + 3 = 6 MST.

Steps	edges	weight	MST vertics
Start	~~AD~~	-	A
1	AD	1	A, D
2	AB	2	A B, D
3	AC	3	A, B, C, D

Algorithm

PRIM(G)
Choose any starting vertex
MST = empty
Mark starting vertex as visited
while MST has fewer then V-1 edges:
Find the minimum-weight edge
Connecting:
visited vertex
↓
unvisited vertex.
Add this edge to MST
Mark the new vertex as visited.

Complexity $O(V^2)$ => adjacent matrix + array

$O(E \log V)$ => adjacency list + binary min heap.

* Kruskal's Algorithm

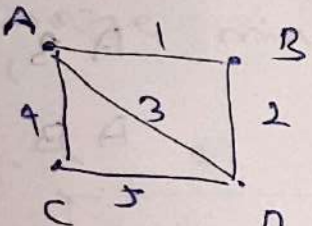
Kruskal's Algorithm is a greed algorithm that construct an MST by considering edges in increasing order of weight & adding an edge if it does not create a cycle.

Note

≡ kruskal . ~~edge~~

Sort edge & smallest edge for start

Ex ≡



$V-1 = 3$

$AB = 1$

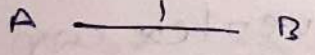
$BD = 2$

$AD = 3$ $\quad CD = 5$

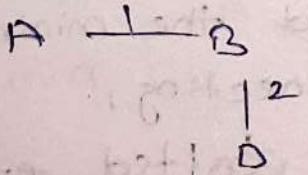
$AC = 4$

sort then select

select A-B = 1 (No cycle then select)

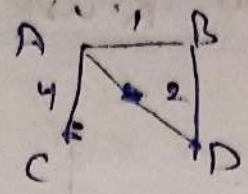


select B-D



A-D Bad cycle create so not selected.

Now A - C



All visited. Stop here.

final edges :: A-B, B-D & AC

1 + 2 + 4 = 7

Algorithm

Kruskal (G)

1. Sort all edges in increasing order of weight.
2. Create a separate set for every vertex.
3. MST = empty
4. for each edge (u, v) in sorted order:
   if FIND(u) != FIND(v):
   Add edge(u, v) to MST
   UNION(u, v)
5. Stop when MST contains v - 1 edges.
6. Return MST.

Complexity

$O(E \log E)$

$\because E < V^2$

So $O(E \log V)$

Difference between Kruskal & Prims.

Feature	Prims	Kruskal
Approach	Vertex-based	Edge-based
Start with	One vertex	All edges
Main operation	Grow one tree	Add smallest safe edge
Sorting edge required?	Not necessarily	Yes
Cycle handling	Naturally avoided by visited set	DSU / Union-find
Common DS	Min Heap / Priority Queue	DSU
Typical complexity	$O(E \log V)$	$O(E \log E)$
Good for	Dense Tree	Sparse tree

Matrix * Dynamic Programming :-

Design technique used to solve complex problems by breaking them into smaller overlapping

Two main Properties

1. Optimal Substructure :-

The optimal solution of the main problem can be constructed from optimal solutions of its subproblems.

2. Overlapping Subproblem :-

The same subproblems occur multiple times, so we store their solutions to avoid repeated computation.

Ex :

- matrix chain Multiplication
- Optimal binary search tree
- Longest Common Subsequence
- 0/1 knapsack.

* Matrix Chain Multiplication

Technique used to determine the optimal parenthesization of a sequence of matrics so that the number of scalar multiplication is minimized without changing the order of matrics.

Recurrence

$$m[i][j] = \min_{i \le k < j} \{ m[i][k] + m[k+1][j] + p_{i-1} p_k p_j \}$$

left chain — Right chain

Time complexity = $O(n^3)$

Space complexity = $O(n^2)$

Rule $(A_1 A_2) A_3 = A_1 (A_2 A_3)$

Due to Associativity, but multiplication cost will not remain same.

So we have to make minimum scaler multiplication

→ we only change parenthesize order not matrics order

Here Note ~~M[i][j] =~~

for $i = j$

$M[i][j] = 0$

So suppose $A_i A_{i+1} \cdots A_j$

$(A_i \cdots A_k)(A_{k+1} \cdots A_j)$

Q. $p = [10, 20, 30, 40]$

matrices: $A_1 = 10 \times 20$

$A_2 = 20 \times 30$

$A_3 = 30 \times 40$

Initial table

	A_1	A_2	A_3
A_1	0	?	?
A_2	–	0	?
A_3	–	–	0

$A_1 A_2 \Rightarrow 10 \times 20 \times 30 = 6000$

$m[i][j] = m[1][2] = 6000$

$A_2 A_3 = 20 \times 30 \times 40 = 24000$

$m[2][3] = 24000$

Table

	A_1	A_2	A_3
A_1	0	6000	?
A_2	–	0	24000
A_3	–	–	0

Note $m[1][3] = m[1][2] + m[3][3] + p_0 p_2 p_3$

$i < k \leq j$

$k = 2$ or 3

$= 6000 + 10 \times 30 \times 40$

① $= 18000$ — ①

& for $k = 3 = m[1][1] + m[2][3] + p_0 p_1 p_3$

$= 0 + 24000 + 10 \times 20 \times 30$

$= 32000$ — ②

minimum (32000, 18000) = 18000

So, m[1][3] = 18000

So minimum Cost = 18000.

$(A_1 A_2) A_3$ optimal parenthesization

Algorithm

```
MATRIX-CHAIN-MULTIPLICATION(P, n)
  for i = 1 to n
      m[i][i] = 0
  for length = 2 to n
      for i = 1 to n - length + 1
          j = i + length - 1
          m[i][j] = ∞
          for k = i to j - 1
              Cost = m[i][k]
                   + m[k+1][j]
                   + P[i-1] * P[k] * P[j]
              if Cost < m[i][j]
                  m[i][j] = Cost
  return m[1][n]
```

Q. Find optimal parentheirzation of a matrix chain product whose sequence of dimension <5, 10, 3, 12, 5, 50, 6>

Sol:-

$$< 5, 10, 3, 12, 5, 50, 6 >$$
$$P_0 \quad P_1 \quad P_2 \quad P_3 \quad P_4 \quad P_5 \quad P_6$$

matrix chain length => n-1 => 7-1 = 6

So two auxillary table m & k

if i = j => 0

m[i,j] = m[1,1] = m[2,2] = m[3,3] = m[4,4]

= m[5,5] = m[6,6] = 0

j → / i ↓	1	2	3	4	5	6
1	0	150	330	405	1655	2010
2		0	360	330	2430	1950
3			0	180	930	1770
4				0	3000	1860
5					0	1500
6						0

i ↓ / k →	1	2	3	4	5	6
1	0	1	2	2	4	2
2		0	2	2	2	2
3			0	3	4	4
4				0	4	4
5					0	5
6						0

① $m[1,2] = \{ m[1,1] + m[2,2] + P_0 P_1 P_2 \}$

$K \Rightarrow 1 \le K < 2$ $\quad 0 + 0 + 5 \times 10 \times 3 = 150$

$K = 1$ ✓

② $m[2,3] = \{ m[2,2] + m[3,3] + P_1 P_2 P_3 \}$

$1 <= 2 \le K < 3$ $= 0 + 0 + 10 \times 3 \times 12$

$\therefore = 2$ $= 360$

(3) $m[3,4] = \{ m[3,3] + m[4,4] + P_2 P_3 P_4 \}$

$3 \le k < 4$ $= 0 + 0 + 3 \times 12 \times 5 = 180$

$(k = 3)$

(4) $m[4,5] = \{ m[4,4] + m[5,5] + P_3 P_4 P_5 \}$

$4 \le k < 5$ $= 0 + 0 + 12 \times 5 \times 50$

$(k = 4)$ $= 3000$

(5) $m[5,6] = \{ m[5,5] + m[6,6] + P_4 P_5 P_6 \}$

$5 \le k < 6$ $= 0 + 0 + 5 \times 50 \times 6$

$(k = 5)$ $= 1500$

(6) $m[1,3]$

$1 \le k < 3$

$k =$

for $k = 1$

$m[1,1] + m[2,3] + P_0 P_1 P_3$

$0 + 360 + 5 \times 18 \times 12 = 960$

for $k = 2$

$m[1,2] + m[3,3] + P_0 P_2 P_3$

$150 + 0 + 5 \times 3 \times 12 = 330$

$\min(960, 330) = 330$

(7) $m[2,4]$

$2 \le k < 4$

k

for $k = 2$

$m[2,2] + m[3,4] + P_1 P_2 P_4$

$0 + 180 + 10 \times 3 \times 5 = 330$

for $k = 3$

$m[2,3] + m[4,4] + P_1 P_3 P_4$

$360 + 0 + 10 \times 12 \times 5$

$=$

$\min(330, 960) = 330$

(8)

$m[3,5]$ = ~~$m[3,$~~ for $k=3$

$3 \le k < 5$ $\quad m[3,3] + m[4,5] + P_2 P_3 P_5$

$0 + 3000 + 3 \times 12 \times 50 = 1500$

$= 4800$

for $k = 4$

$m[3,4] + m[5,5] + P_2 P_4 P_5$

$180 + 0 + 3 \times 5 \times 50$

$= 930$

$\min(930, 4800) = 930$

(9) $m[4,6]$ for $k = 4$

$4 \le k < 6$ $\quad m[4,4] + m[5,6] + P_3 P_4 P_6$

$0 + 1500 + 12 \times 5 \times 6$

$= 1860$

for $k = 5$

$m[4,5] + m[6,6] + P_3 P_5 P_6$

$1500 + 0 + 12 \times 50 \times 6$

$= 6600$

$\min(1860, 6600) = 1860$

like wise

$m[1,4] = 405$

$m[1,5] = 1655$

$m[1,6] = 2010$

$m[2,5] = 2630$

$m[2,6] = 1950$

$m[3,6] = 1770$

So $m[1,6] = 2010$ It will be the minimum number of scalar multiplication

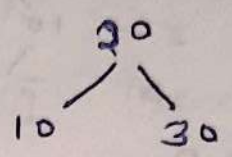
So $\boxed{(P_1 P_2)(P_3 P_4)(P_5 P_6)}$

Optimal Binary search Tree

Binary search Tree

Left key < Root key < Right key

10, 20, 30 =>



Optimal BST is a BST arranged so that the expected search cost is minimum.

→ The best BST depends on the search freq./ dependencies/probability)

Success Search ⇒ key found at comparison.

Unsuccessfull search ⇒ key not found.

p = successful search

q = unsuccessfull search.

→ Consider every possible key as the root of a subproblem & choose the root giving minimum cost

e[i][j] DP state

minimum expected search cost for key K_i to K_j

* Base case ⇒ $i = j+1$

$q_{i-1} = e[i][i-1]$ or $w[i][i-1]$

** $w[i][j] = w[i][j-1] + p_j + q_j$

if K_r is root

$$e[i][j] = \min_{i \le r \le j} (e[i][r-1] + e[r+1][j] + w[i][j])$$

Left + Right + weight of full.

Q=

Key index (i)	0	1	2
key (k)	10	12	20
frequency (f)	34	8	50

base case $i=j = freq[i]$ (Level 1)

if $j<i$ $Cost[i][j] = 0$

$Cost[0][0] = freq[0] = 34$

$Cost[1][1] = freq[1] = 8$

$Cost[2][2] = freq[2] = 50$

So table

i \ j	0	1	2
0	34	—	—
1	0	8	—
2	0	0	50

for length 2 (Level 2) [0,1] & [1,2] r = Level

Range [0,1] Key = 10, 12 $\Sigma freq = 38+4 = 42$

if root = 10 $Cost[i][r-1] + Cost[r+1][j] + \Sigma freq.$

$Cost = Cost[0][\overset{i=r\Rightarrow 0}{0}] + Cost[1][1] + 42$

(Key index. 0) $= r$ $= 0 + 8 + 42 = 50$

if Root = 12 $r=j \Rightarrow 0$

$Cost = Cost[0][0] + Cost[2][1] + 42$

Key index 1

$= 34 + 0 + 42 = 76$

min = 50

Root key is 10.

Now in Renge [1, 2] key 12, 20

$\Sigma freq = 8+50 = 58$

$i = r \Rightarrow 0$

if root = 12 index $1 = r$

$Cost[i, r-1] + Cost[r+1, j] + \Sigma freq.$

$0 + Cost[2,2] + 58$

$\Rightarrow 0 + 50 + 58$

$\Rightarrow 108$

$j = r = 2 = 1 0$

if root = 20 index $r = 2$

$Cost[i, r-1] + Cost[r+1, j] + \Sigma freq.$

$= Cost[1,1] + 0 + 58$

$= 8 + 58 = 66$

$\min(108, 66) = 60$

So root is key 20

So updated table

i/j	0	1	2
0	34	50	=
1	0	8	66
2	0	0	50

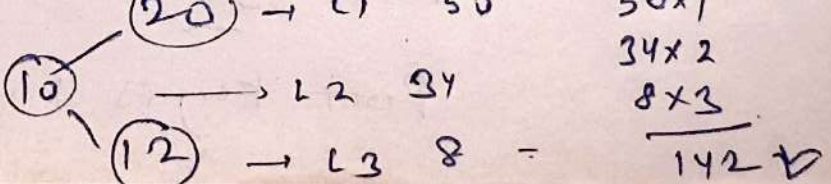
for $l = 3$ Renge [0, 2]

if root = 10
$r = 0$
$0 + Cost[1,2] + 92$
$\Rightarrow 66 + 92$
$\Rightarrow 158$

if root = 12
$r = 1$
$Cost[0][0] + Cost[2][2] + 92$
$34 + 50 + 92$
$= 176$

if root = 20
$r = 2$
$Cost[0,1] + 0 + 92$
$= 50 + 0 + 92$
$= 142$

minimum is 142 So Root is 20.



*Longest Common Subsequence :- (LCS)

Checks Relative order of character but its not necessary to be continuous character

Eg:- ACE ✓ ABCDE
ABD ✓ _ _ _

Substring → continuous ch.
Subsequence → no need of continuous. but order same.

X = ABCDEF Y = AEDFHR

Longest Common Subsequence = AED & AEF

DP state ⇒ L[i][j]
↙ first string ↘ second string

Base Case ⇒ L[i][0] = 0
L[0][j] = 0

compare X[i-1] & Y[j-1] (Left & Top)

if X[i-1] = Y[j-1]
then
L[i][j] = L[i-1][j-1] + 1
(diagonal +1)

X[i-1] ≠ Y[j-1]
then
L[i][j] = max (Left & Top)
direction of which one is selected.

Eg X = A B C B D A B (if top = left : then choose Left)

Y = B D C A B A

rule if string match ⇒ diagonal +1

not matched ⇒ max(Top, Left)

x \ Y	ϕ	B	D	C	A	B	A
ϕ	0	0	0	0	0	0	0
A	0	0←	0←	0←	1↖	1←	1↖
B	0	1↖	1←	1←	1←	2↖	2←
C	0	1↑	1←	2↖	2←	2←	2←
B	0	1↖	1←	2↑	2←	3↖	3←
D	0	1↑	2↖	2←	2←	3↑	3←
A	0	1↑	2↑	2←	3↖	3←	4↖
B	0	1↑	2↑	2←	3↑	4↖	4←

First Row.

A v/s B No match → left value

A v/s D No match → left

A v/s C No match → left

A v/s A match diagonal (0+1) = 1↖

A v/s B No match → left

A v/s A matched → diagonal +1 = 1↖

Similarly for all row.

L[8][6] = 4 longest common subsequence

BCBA or BCAB

Algorithm

```
LCS (X, Y, m, n)
  create L[m+1][n+1]
  for i = 0 to m
    L[i][0] = 0
  for j = 0 to n
    L[0][j] = 0
  for i = 1 to m
    for j = 1 to n
      if X[i-1] == Y[j-1]
        L[i][j] = L[i-1][j-1] + 1
      else
        L[i][j] = max(L[i-1][j], L[i][j-1])
  return L[m][n]
```

Time Complexity = $O(mn)$

Space Complexity = $O(mn)$

* 0/1 knapSack Problem

The 0/1 knapsack problem is an optimization problem in which each item can either be selected completely or not selected at all. The objective is to maximize the total profit without exceeding the given knapsack capacity.

→ Total maximum Profit

→ Total weigth $\leq W$

0 ↳ don't take & 1 ↳ take

dp[i][w]

use first i item, ~~maximum~~ find maximum profit under maximum weight capacity.

Base Case

i = 0 w = 0

dp[0][w] = 0 dp[i][0] = 0

so first row & column = 0

if weight[i] > w

Yes ↓ No ↓

dp[i][w] = dp[i-1][w]

(mean skip current item)

Don't choose dp[i] → dp[i-1][w]

Choose

profit[i] + dp[i-1][w - weight[i]]

we choose

max (dp[i-1][w] , profit[i] + dp[i-1][w - weight[i]])

Algorithm

```
0-1_knapSack(weight, profit, n, W)
create dp[n+1][W+1]
for i = 0 to n
    dp[i][0] = 0
for w = 0 to W
    dp[0][w] = 0
for i = 1 to n
    for w = 1 to W
        if weight[i] > w
            dp[i][w] = dp[i-1][w]
        else
            notTake = dp[i-1][w]
            take = profit[i] + dp[i-1][w-weight[i]]
        dp[i][w] = max(take, notTake)
return dp[n][W]
```

Time Complexity : $O(nW)$

space Complexity : $O(nW)$

(pseudo polynomial time)

Q

Item	weight	Profit
1	1	6
2	2	10
3	3	12

given capacity = W = 5

So first Create table.

item \ Capacity	0	1	2	3	4	5
0 item	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Row 1, item 1

Weight = 1 Profit = 6

Dont take = 0

take = 6 + dp[0][0] = 6

max = 6

Row 2 item 2

weight = 2, Profit = 10

(1 ≠ 2) Capacity 1

dp[2][1] = 6

dp[1][1] = 6

Capacity 2 (2 = 2).

dp[1][2] = 6 Dont take

take = 10 + dp[1][0] = 10

max = 10

Capacity 3

Not ⇒ dp[1][3] = 6

take ⇒ dp[0][1] + 10 ⇒ 16

max = 16

then all will be skip part because W < weight[i]

Row 3 item 3

weight = 3 , Profit = 12

Capacity 1
don't fit
skip

Capacity 2
don't fit
skip

Capacity 3 i=3, w=3
fit
don't take = dp[2][3]
= 16

take = 12 + dp[2][0]
= 12

max = 16

Capacity 4
don't take
dp[2][4] = 16

Take 12 + dp[2][1]
= 12+6 = 18

max = 18 ✓

Capacity 5 (w = 5-3 = 2)
don't take
dp[2][5] = 16

Take =
12 + dp[2][2]
12 + 10 = 22

(max = 22)

Which items selected

item 2 & 3